Digital Object Identifier 10.1109/ACCESS.2026.XXXXXXX

HMRC: Heterogeneous Multi-Representative Centroids for Improved Vector Database Routing

KISHORE PULAGAM[1]
[1]Northwestern University, Evanston, IL, USA and PayPal Data Services

I. INTRODUCTION

Vector databases have become the retrieval backbone for a growing class of applications: retrieval-augmented generation [11], semantic search, recommendation engines, and more. At scale, these systems cannot afford to compare every query against every stored vector. They instead partition the corpus into segments and route incoming queries to the most promising ones before performing exhaustive search within each. Routing relies on comparing the query to a compact summary of each segment, almost always the mean of its vectors.

This two-phase architecture turns on a simple assumption: the mean centroid captures enough about a segment's content that a nearby query will be routed correctly. The assumption holds when segments are semantically coherent. It breaks, sometimes badly, when they are not.

Heterogeneous segments, meaning segments whose vectors span multiple unrelated topics, arise through mechanisms that have little to do with retrieval quality. Documents ingested in time-ordered batches reflect whatever topics appeared that day. Multi-tenant systems group vectors by customer, not by meaning. Incremental updates append new documents to existing segments rather than trigger full re-clustering [12]. Even k-means itself, the standard tool for creating segments, produces imperfect partitions, especially at scale [13].

The consequence is a centroid that lies in the geometric middle of topics it should represent, close to none of them. A query about machine learning might route to the wrong segment because its true segment also contains unrelated medical or legal content, pulling the centroid away from the ML region. We quantify this effect on two passage retrieval datasets and find that mixing three topics per segment is enough to drop routing recall by 35 percentage points.

The fix we explore is simple. Rather than summarize each segment with a single mean, we run k-means clustering within the segment and use the resulting cluster centers as multiple representatives. This idea has precedent in the clustering literature. The CURE algorithm [1] uses scattered representative points to capture non-spherical cluster geometry, and the kRt routing method in Pyramid [2] sub-clusters shards for billion-scale distributed search. What has not been done, to our knowledge, is applying multi-representative centroids specifically to the segment routing problem in IVF-style indexes where segments are fixed partitions that cannot be re-clustered. HMRC fills that gap.

Our contributions:

1) A quantification of the heterogeneous segment problem on two datasets: mixing three topics per segment drops routing recall from 100% to 60–64%.
2) HMRC, a method that generates multiple representative centroids per segment via intra-segment k-means clustering.
3) Experimental evidence that three representatives per segment recover most of the lost recall (reaching 79–81%) at 0.6% storage overhead. On both datasets, HMRC searching 10 segments outperforms the baseline searching 30.
4) Validation on realistic time-batched segments showing the improvement transfers beyond synthetic setups.

II. BACKGROUND AND RELATED WORK

A. IVF INDEXING AND CENTROID-BASED ROUTING

Inverted File (IVF) indexing, as implemented in FAISS [3], Milvus [4], and most production vector databases [10], partitions the corpus into clusters using k-means and represents each cluster with its centroid. At query time, the system compares the query vector to all centroids, selects the top-

nprobe clusters, and searches exhaustively within them. The centroid comparison is cheap (typically a few hundred distance calculations), while within-cluster search dominates runtime. This architecture reduces search complexity from $O(n)$ to $O(n/m + m)$, where n is the corpus size and m the number of segments.

The critical design choice is how segments are represented. Every major implementation uses the arithmetic mean. This works well when segments correspond to semantically tight clusters. It works less well when they do not, and the standard remedy, increasing *nprobe*, trades recall for latency linearly.

Within-cluster search itself can be accelerated through compression techniques like product quantization (PQ) [14], which encodes vectors into compact codes for faster distance computation, or through graph-based sub-indexes [15]. These optimizations operate downstream of routing and are orthogonal to the centroid representation problem we address.

B. MULTI-PROBE SEARCH

The *nprobe* parameter controls how many clusters are searched per query. Increasing it is the standard way to recover recall lost to imperfect routing. But the cost is proportional: searching three clusters takes roughly three times longer than searching one. Multi-probe LSH [7] applies a related idea in hash-based indexes, probing nearby hash buckets to catch boundary cases. Our experiments show that HMRC achieves better routing quality than tripling *nprobe*, at the same centroid comparison cost (Section V-C).

C. VECTOR REPLICATION: SOAR / SCANN

Google's SOAR algorithm [5] takes a different approach: it assigns vectors to multiple clusters so that a vector has backup clusters that can catch queries the primary cluster misses. SOAR's key insight is selecting secondary assignments whose residuals are orthogonal to the primary residual, maximizing the chance that at least one assignment performs well for any query direction. The method improves recall at 2–3$\times$ vector storage cost.

HMRC differs in what it replicates. SOAR copies vectors across clusters, adding a duplicate vector entry for each secondary assignment. HMRC copies centroids, a far smaller data structure. At the scale of our experiments (200 segments, 500 vectors each), HMRC-3 adds 400 centroids (600 KB), while a SOAR-like approach would duplicate a substantial fraction of the 100,000 vectors themselves. At larger scales the gap widens further: for a system with 10,000 segments of 1,000 vectors each, HMRC-3 adds 20,000 centroids ($\sim$30 MB at 384 dimensions), whereas vector replication at 2$\times$ would add up to 10 million vector copies ($\sim$15 GB). The mechanisms target different bottlenecks and could in principle be combined.

D. MULTI-REPRESENTATIVE CLUSTERING: CURE

The CURE algorithm [1] represents each cluster with multiple scattered points rather than a single centroid, enabling it to handle non-spherical cluster geometries. CURE was designed as a general-purpose data analysis tool and has not been applied to the two-phase query routing setting of IVF indexes. Our use of multiple representatives is conceptually similar but operates within a different framework: we do not control how segments are formed (they arise from operational constraints), and our goal is routing accuracy, not clustering quality per se.

E. SUB-CLUSTER ROUTING: KRT / PYRAMID

The closest prior work to HMRC is the kRt routing method introduced alongside the Pyramid partitioning system [2], published at VLDB 2025. kRt sub-clusters the points within each shard using hierarchical k-means and routes queries to the shards whose sub-cluster centers are nearest. There are substantive differences. Pyramid targets billion-scale distributed search with graph-partitioned shards, an architecture quite different from the segment-based IVF indexes HMRC addresses. kRt uses hierarchical k-means trees or HNSW over sub-centers, adding structural complexity; HMRC uses flat k-means with a simple nearest-representative lookup. The more relevant difference is that Pyramid assumes shards are optimally partitioned by graph algorithms. HMRC specifically addresses segments that are *not* well-partitioned, such as the time-batched, multi-tenant, and append-only partitions common in enterprise systems. We are also not aware of work quantifying the routing degradation that heterogeneous segments cause, which is the starting point of our contribution.

F. GRAPH-BASED AND DISK-RESIDENT METHODS

HNSW [6] builds navigable small-world graphs and avoids centroids entirely. This sidesteps the heterogeneity problem but introduces different tradeoffs: higher memory overhead (storing graph edges for every vector), difficulty distributing the index, and limited support for the segment-based data management patterns that production systems rely on for updates, deletions, and multi-tenancy. DiskANN [12] and SPANN [16] extend nearest neighbor search to disk-resident settings, using clustering for coarse quantization and storing posting lists on SSD. These systems face the same centroid representation challenge we address, though their primary focus is on I/O optimization rather than routing quality.

III. PROBLEM: CENTROID DEGRADATION IN HETEROGENEOUS SEGMENTS

A. DEFINITIONS

We define a *segment* as a fixed partition of a vector corpus created by operational constraints rather than by semantic clustering. Segments may arise from ingestion batches (all documents arriving in a given time window), tenant boundaries (all documents belonging to a given customer), storage shard assignments, or append-only growth where new documents are added to existing segments to avoid re-indexing. The key characteristic is that segments are not optimized for routing; they may, and typically do, contain vectors from multiple semantic domains.

TABLE 1. Routing Recall@10 degrades with segment heterogeneity.

Segment Type	MS MARCO	NQ	Avg. Drop
Coherent (1 topic)	100.0%	100.0%	—
Semi-3 (3 topics)	64.4%	60.2%	−37.7%
Semi-5 (5 topics)	49.2%	43.8%	−53.5%
Random	14.6%	13.6%	−85.9%

Let $\mathcal{D} = \{\mathbf{v}_1, \ldots, \mathbf{v}_n\}$ be a corpus of n vectors in $\mathbb{R}^d$, partitioned into m segments $\{S_1, \ldots, S_m\}$. Each segment S_j is represented by its mean centroid:

$$\mathbf{c}_j = \frac{1}{|S_j|} \sum_{\mathbf{v} \in S_j} \mathbf{v} \tag{1}$$

Given a query $\mathbf{q}$, routing selects the top-k segments by centroid distance:

$$\text{Route}(\mathbf{q}, k) = \underset{j \in [m]}{\text{argmin-k}} \, \|\mathbf{q} - \mathbf{c}_j\| \tag{2}$$

Routing Recall@k measures the fraction of queries whose ground-truth segment appears in the top-k.

B. WHY MEAN CENTROIDS FAIL

Consider a segment $S = S_A \cup S_B$ containing vectors from two topic clusters with means $\boldsymbol{\mu}_A$ and $\boldsymbol{\mu}_B$. The segment centroid $\mathbf{c} = \frac{|S_A|}{|S|}\boldsymbol{\mu}_A + \frac{|S_B|}{|S|}\boldsymbol{\mu}_B$ lies on the line between $\boldsymbol{\mu}_A$ and $\boldsymbol{\mu}_B$, potentially far from both. A query near topic A satisfies $\|\mathbf{q} - \mathbf{c}\| > \|\mathbf{q} - \boldsymbol{\mu}_A\|$. If any other segment has a centroid closer to topic A's region, the query routes incorrectly.

C. EMPIRICAL VALIDATION

We validated this on two passage retrieval corpora: 100K passages from MS MARCO [17] embedded with all-MiniLM-L6-v2 [18] (384 dimensions), and 100K passages from Natural Questions [19] embedded with the same model. We created 200 segments of 500 vectors each under four conditions: *Coherent* (segments formed by k-means clustering), *Semi-t* (each segment mixes vectors from t random topic clusters), and *Random* (uniform random assignment). We sampled 2,000 queries from each corpus. The degradation pattern is shown in Fig. 1.

The pattern is consistent across datasets. Three topics per segment, a mild level of heterogeneity easily produced by a single day's news articles or a moderately diverse customer account, is sufficient to lose over a third of routing accuracy.

IV. HMRC: OUR APPROACH

A. INDEX CONSTRUCTION

For each segment S_j, we run k-means clustering with r clusters (we use $r = 3$ as the default). The resulting cluster centers become the segment's representatives:

$$R_j = \text{KMeans}(S_j, r).\text{centers} \tag{3}$$

All representatives are stored in a flat structure with a mapping from each representative to its segment ID. For segments with fewer than $2r$ vectors, we fall back to the mean, since k-means would produce unreliable results with so few points.

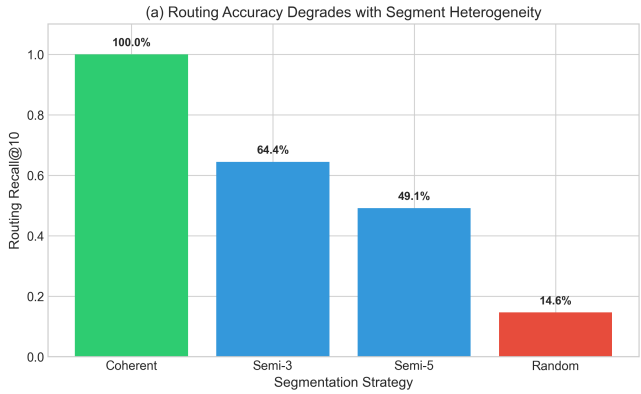

FIGURE 1. Routing Recall@10 degrades as segments become more heterogeneous (MS MARCO, 100K passages, 200 segments). Coherent segments achieve perfect recall; mixing three topics drops recall to 64.4%.

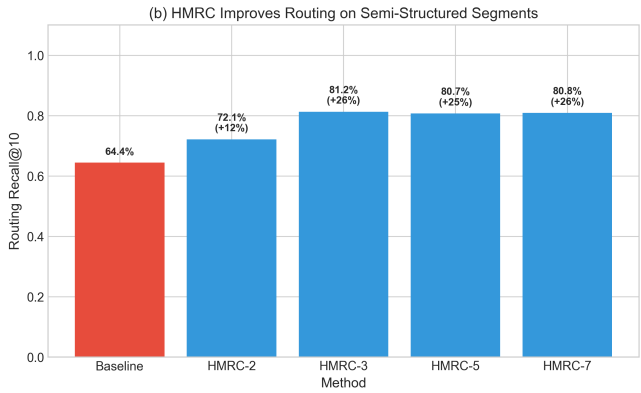

FIGURE 2. HMRC improves routing on Semi-3 segments (MS MARCO). HMRC-3 achieves 81.2% Recall@10, a 26% relative improvement over the 64.4% baseline. Performance plateaus beyond $r = 3$.

B. QUERY ROUTING

Given a query $\mathbf{q}$, HMRC replaces the single-centroid distance with a minimum distance over representatives. The routing decision becomes:

$$\text{Route}_{\text{HMRC}}(\mathbf{q}, k) = \underset{j \in [m]}{\text{argmin-k}} \min_{\mathbf{c} \in R_j} \|\mathbf{q} - \mathbf{c}\| \tag{4}$$

That is, each segment's distance to the query is determined by its *closest* representative rather than its mean. This ensures that a segment containing topic A content will have at least one representative near topic A, regardless of what other topics the segment contains. After computing distances to all $r \times m$ representatives, we deduplicate by segment ID and return the top-k unique segments ranked by their closest representative.

C. COHERENT SEGMENT SAFETY

When a segment is already coherent, containing vectors from a single topic, k-means produces r centroids that are

TABLE 2. Build time, memory, and routing overhead. Measured on MS MARCO 100K, 200 segments of 500 vectors, 384 dims. NQ timings are within 5% of these values.

Method	Centr.	Mem. (KB)	% Vec.	Build (s)	Per-Seg (ms)	Route (μs/q)
Baseline	200	300	0.20	0.04	—	71
HMRC-2	400	600	0.40	0.77	3.6	142
HMRC-3	600	900	0.60	0.88	4.1	200
HMRC-5	1000	1500	1.00	1.21	6.2	357
HMRC-7	1400	2100	1.40	1.32	6.5	500

all close to each other, covering the same semantic region. Routing works because any of these centroids will be near queries from that topic. Our experiments confirm this: HMRC achieves 100% recall on coherent segments in both datasets, matching the baseline exactly. There appears to be no downside when segments are already well-structured.

D. COMPLEXITY AND OVERHEAD

Storage. HMRC stores $r \times m$ centroids instead of m. For 200 segments with 384-dimensional float32 vectors, the baseline requires 300 KB of centroid storage and HMRC-3 requires 900 KB. Both figures are negligible compared to the 146.5 MB of vector data. The centroid overhead is 0.6% of vector storage.

Query routing time. Routing requires $r\times$ more distance computations to centroids. We measured per-query routing latency on 2,000 queries across 200 segments: the baseline routes a query in 71 μs, while HMRC-3 takes 200 μs, a 2.8$\times$ increase that remains well under one millisecond. For context, the within-segment search that follows routing typically takes 10–100 ms depending on segment size and compression, making the routing step a negligible fraction of total query latency regardless of method.

Build time. K-means within each segment adds build-time cost. On MS MARCO (200 segments, 500 vectors each), HMRC-3 builds in 876 ms total, with per-segment k-means averaging 4.1 ms. The baseline builds in 42 ms. This is a one-time offline cost and takes under one second regardless.

E. SELECTING R

How many representatives should each segment have? Our ablation (Section V-F) shows that performance plateaus at $r = 3$ for segments mixing three topics: recall reaches 81.2% at $r = 3$ and does not meaningfully improve at $r = 4$, 5, 7, or 10, as shown in Fig. 5. This suggests r should roughly match the expected number of topics per segment. For workloads where heterogeneity is unknown, $r = 3$ is a conservative default that captures most of the available improvement at minimal overhead. Practitioners who suspect higher diversity can increase r, though the marginal benefit diminishes.

V. EXPERIMENTS

A. SETUP

Datasets. We use two passage retrieval corpora drawn from standard benchmarks used in information retrieval evaluation [20]:

- *MS MARCO* [17]: 100,000 passages from the Microsoft Machine Reading Comprehension dataset, embedded with all-MiniLM-L6-v2 [18] (384 dimensions).
- *Natural Questions (NQ)* [19]: 100,000 passages from Google's Natural Questions, embedded with the same model.

Both are standard benchmarks in information retrieval research. Using the same embedding model across datasets isolates the effect of corpus content from model choice.

Segmentation. We create 200 segments of 500 vectors each under four conditions: Coherent (k-means clustering), Semi-t (each segment draws from t random topic clusters), Time-Batched (consecutive assignment simulating ingestion order), and Random (uniform random assignment). For Semi-t experiments, we pre-cluster the corpus into 100 topic clusters and then assign mixtures of these topics to segments.

Queries. 2,000 queries sampled uniformly from each corpus. Ground truth is the segment containing the query vector. We report Routing Recall@k, defined as the fraction of queries whose true segment appears in the top-k routed segments.

Implementation. All experiments run in Python using NumPy and scikit-learn [21]. K-means within segments uses scikit-learn's KMeans with 3 initializations and 100 maximum iterations. All vectors are L2-normalized before indexing. Experiments were conducted on an Apple M-series processor; timings are representative of single-threaded performance on commodity hardware.

Figures. All figures present MS MARCO results. NQ results, which follow the same patterns, are reported alongside MS MARCO in each table.

TABLE 3. HMRC on Semi-3 segments (Recall@10).

Method	MS MARCO	NQ	Overhead
Baseline	64.4%	60.2%	1$\times$
HMRC-2	72.1%	70.2%	2$\times$
HMRC-3	81.2%	78.8%	3$\times$
HMRC-5	80.7%	79.0%	5$\times$
HMRC-7	80.9%	78.9%	7$\times$

B. MAIN RESULTS

HMRC-3 improves routing recall by 26% on MS MARCO and 31% on NQ relative to the baseline. The improvement is larger on NQ, where the baseline starts lower, suggesting that HMRC helps more when the baseline struggles more. Performance plateaus at $r = 3$, indicating that additional representatives beyond the number of mixed topics provide no measurable benefit.

C. COMPARISON WITH MULTI-PROBE BASELINE

A natural objection to HMRC is that one could simply search more segments instead of improving the centroid representation. HMRC-3 compares against 600 centroids (3

TABLE 4. Extended-k comparison on Semi-3 segments.

k	Baseline		HMRC-3	
	MSMARCO	NQ	MSMARCO	NQ
1	13.3%	12.2%	26.9%	27.4%
3	32.6%	31.0%	56.7%	55.9%
5	45.5%	43.5%	72.7%	70.0%
10	64.4%	60.2%	81.2%	78.8%
15	72.7%	67.5%	82.9%	80.7%
20	77.1%	72.6%	84.3%	81.9%
30	81.1%	77.9%	86.2%	84.0%

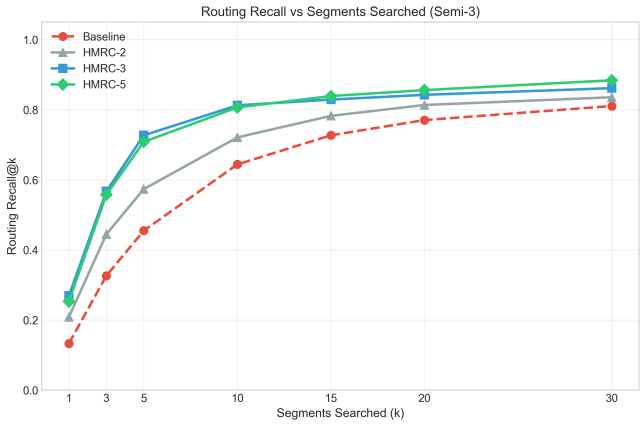

FIGURE 3. Routing Recall at various k values vs segments searched on Semi-3 segments (MS MARCO). HMRC-3 searching 10 segments (81.2%) surpasses the baseline searching 30 segments (81.1%), achieving equivalent routing quality with 3x fewer segments examined.

per segment $\times$ 200 segments) rather than 200. The question is whether spending the same effort on searching more segments, effectively increasing k, would yield equal or better recall.

The key comparison, illustrated in Fig. 3: HMRC-3 at k=10 (searching 10 segments) outperforms the baseline at k=30 (searching 30 segments) on both datasets. On MS MARCO, 81.2% vs 81.1%. On NQ, 78.8% vs 77.9%. HMRC achieves comparable or slightly better routing quality while searching three times fewer segments. This is not a marginal difference in centroid comparisons; it translates directly to 3$\times$ less within-segment search, which is the dominant component of query latency in IVF-based systems [3].

At lower k values the advantage is even more pronounced. At k=5, HMRC-3 reaches 72.7% on MS MARCO vs the baseline's 45.5%, a 60% relative improvement. Systems that route to few segments for latency reasons benefit most from HMRC. Section V-D extends this analysis to a real FAISS IVF index, where the same pattern holds for end-to-end retrieval recall and translates into a measurable wall-clock latency improvement.

D. VALIDATION IN FAISS INDEXIVFFLAT

The extended-k comparison in Section V-C runs against our reference NumPy implementation. To verify the result transfers to a production-grade IVF system, we re-ran the comparison inside FAISS [3]. We built two `IndexIVFFlat`

TABLE 5. FAISS IndexIVFFlat: Recall@10 and search latency on Semi-3 segments. Bold rows mark the matched-recall comparison.

Method	*nprobe*	Recall@10		Latency (μs/q)	
		MSM	NQ	MSM	NQ
Baseline	10	0.549	0.488	120	117
Baseline	20	0.687	0.617	233	236
Baseline	30	0.736	0.684	366	342
HMRC-3	10	0.651	0.587	46	46
HMRC-3	**20**	**0.738**	**0.682**	**87**	**86**
HMRC-3	30	0.769	0.722	124	123

indexes that differ only in their coarse quantizer. The baseline uses one mean centroid per segment, giving 200 inverted lists. HMRC-r uses the r representatives per segment as separate inverted lists, giving 200r lists. Each vector is placed in the inverted list whose centroid is nearest within its assigned segment, in both indexes, so the underlying segment boundaries remain identical. We hold 2,000 queries out of the 100K corpus to compute brute-force ground-truth nearest neighbors, sweep *nprobe* $\in \{1, 3, 5, 10, 15, 20, 30\}$, and measure routing recall, end-to-end Recall@10 against ground truth, and per-query search latency on a single thread.

FAISS routing recall closely tracks the NumPy reference. HMRC-3 at *nprobe*=10 reaches 78.2% on MS MARCO and 75.7% on NQ, within three points of the values reported in Section V-C. The small gap reflects the held-out queries, which slightly thin each segment.

The routing improvement carries through to end-to-end retrieval. HMRC-3 at *nprobe*=20 reaches Recall@10 of 73.8% on MS MARCO and 68.2% on NQ, statistically matching the baseline's 73.6% and 68.4% at *nprobe*=30 (Table 5, Fig. 4). The metric users actually care about, whether the true nearest neighbor appears in search results, moves with the routing-level improvement.

Latency moves too, and in the right direction. HMRC distributes the corpus across more, smaller inverted lists, so FAISS searches less work per probe. HMRC-3 at *nprobe*=20 completes in 87 μs/query on MS MARCO. The baseline at *nprobe*=30 takes 366 μs/query for comparable recall. That is a 4.2$\times$ reduction in search latency at matched recall (4.0$\times$ on NQ), a wall-clock benefit not visible in the centroid-cost analysis of Section V-C.

The latency benefit deserves a careful framing. HMRC is not intrinsically faster at fixed *nprobe*; the coarse quantizer has more centroids to compare, which costs work. The advantage emerges because HMRC reaches a target recall at a lower *nprobe* than the baseline needs. A lower *nprobe* means less within-list scanning, and within-list scanning dominates total query time. The story is consistent with Section V-C, but stronger, because it shows up in real wall-clock time inside an off-the-shelf vector search library rather than only in centroid comparisons.

E. GENERALIZATION ACROSS SEGMENT TYPES

On coherent segments, HMRC maintains 100% recall on both datasets. It does not degrade routing when segments are

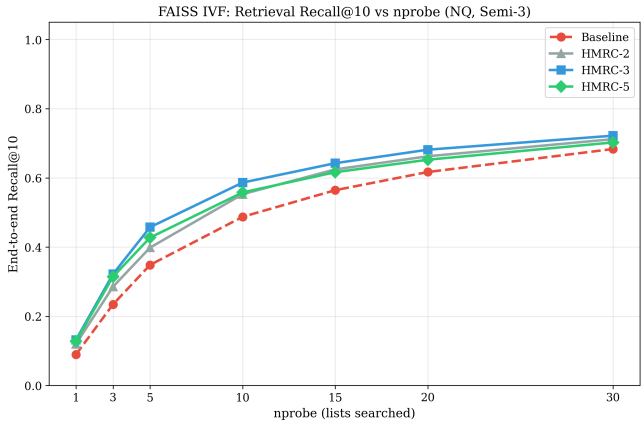

FIGURE 4. End-to-end Recall@10 vs *nprobe* in FAISS IndexIVFFlat (MS MARCO, Semi-3 segments). HMRC-3 at *nprobe*=20 matches baseline retrieval quality at *nprobe*=30, with 4.2× lower search latency.

TABLE 6. Recall@10 across segmentation strategies.

Dataset	Method	Coherent	Semi-3	Random
MSMARCO	Baseline	100.0%	64.4%	14.6%
	HMRC-3	100.0%	81.2%	16.2%
	HMRC-5	100.0%	80.7%	22.2%
NQ	Baseline	100.0%	60.2%	13.6%
	HMRC-3	100.0%	78.8%	14.8%
	HMRC-5	100.0%	79.0%	19.6%

already well-structured. On random segments, improvements are modest (14.6% → 16.2% on MS MARCO) because there is no topic structure for k-means to discover. The method provides value when there is internal structure to exploit, and stays out of the way when there is not.

F. ABLATION: NUMBER OF REPRESENTATIVES

We varied r from 1 (baseline) to 10 on Semi-3 segments. On both datasets, recall climbs sharply from r=1 to r=3, then plateaus. On MS MARCO: 64.4% → 72.1% → 81.2% → 81.4% → 80.7% → 80.9% → 81.4% for $r = 1, 2, 3, 4, 5, 7, 10$. On NQ the shape is the same: 60.2% → 70.2% → 78.8% → 78.7% → 79.0% → 78.9% → 79.8%. The consistency across datasets suggests that $r = 3$ is a reasonable default for segments mixing a small number of topics. Storage overhead grows linearly with r, so the plateau at $r = 3$ means practitioners can capture nearly all the benefit at minimal cost.

G. ABLATION: HETEROGENEITY LEVEL

HMRC provides gains at every level of heterogeneity, and the relative improvement tends to grow as heterogeneity increases. The pattern holds across both datasets. At 10 topics per segment, where the baseline has degraded to ∼30%, HMRC still recovers roughly 10 percentage points.

H. ABLATION: SEGMENT SIZE

Smaller segments benefit most. At segment size 100, HMRC nearly doubles routing recall on MS MARCO and more than doubles it on NQ. This makes sense: smaller segments have less capacity to absorb topic diversity, so their mean centroids are more severely distorted by mixing. Larger segments dilute the effect of any single off-topic subset, making the mean more stable (though still suboptimal).

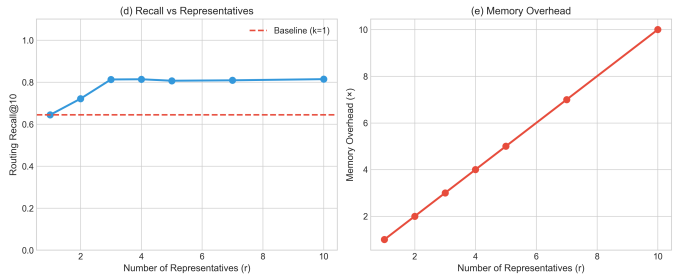

FIGURE 5. Routing recall vs number of representatives on Semi-3 segments.

TABLE 7. HMRC-3 gains across heterogeneity levels (Recall@10).

Topics /Seg	Baseline MSM	Baseline NQ	HMRC-3 MSM	HMRC-3 NQ	Rel. Gain
1	100.0	100.0	100.0	100.0	0%
2	73.5	67.8	80.0	76.6	9–13%
3	64.4	60.2	81.2	78.8	26–31%
5	49.2	43.8	62.7	62.6	27–43%
7	40.0	37.4	48.9	49.3	22–32%
10	30.9	29.2	40.5	40.2	31–38%

I. REALISTIC TIME-BATCHED SEGMENTS

The experiments above use synthetic topic mixing. A fair question is whether real ingestion-order segments exhibit enough heterogeneity for HMRC to help. To test this, we sorted each corpus by document ID (a proxy for arrival order) and assigned consecutive blocks of 500 vectors to segments, simulating daily batch ingestion with no topic-aware partitioning (Fig. 7).

The baseline recall on time-batched segments is substantially worse than on Semi-3 segments, confirming that natural ingestion order creates high heterogeneity. On MS MARCO, HMRC-5 nearly doubles baseline recall (35.1% → 65.0%), and even HMRC-3 yields a 52% relative improvement. NQ presents a harder case: its passages are drawn from Wikipedia, which has a more uniform semantic distribution, and document-ID order does not mirror topical structure as strongly. Still, HMRC-5 reaches 31.9% from a 21.8% baseline, a 46% relative gain, and using higher values of r may be warranted for corpora with this level of dispersion.

The silhouette score for time-batched segments (−0.044 on MS MARCO, −0.047 on NQ) is nearly identical to that of Semi-3 segments (−0.044 and −0.051 respectively), confirming that time-batched partitions are about as heterogeneous as synthetically mixed ones. This suggests that the synthetic mixing used in our other experiments is a reasonable proxy for the heterogeneity that arises in practice.

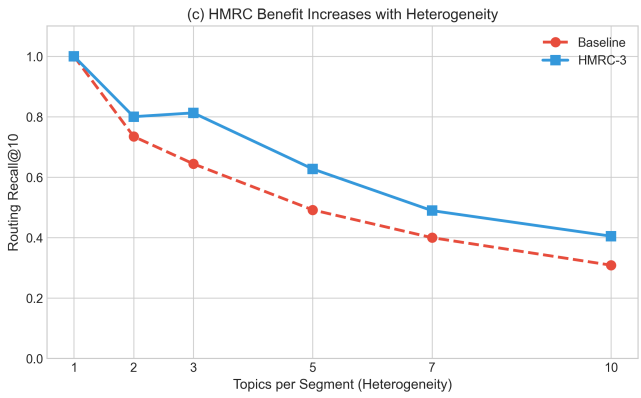

FIGURE 6. HMRC-3 benefit increases with segment heterogeneity (MS MARCO). The gap between baseline and HMRC-3 widens as the number of topics per segment grows, showing HMRC helps most when the problem is worst.

TABLE 8. HMRC-3 across segment sizes (Recall@10 on Semi-3).

Seg.	Baseline		HMRC-3		Rel.
Size	MSM	NQ	MSM	NQ	Gain
100	36.5	31.9	71.9	72.0	97–126%
250	49.5	44.8	77.4	76.7	56–71%
500	64.4	60.2	81.2	78.8	26–31%
1000	69.0	67.3	76.8	76.7	11–14%
2000	71.4	68.5	74.7	72.2	5%

VI. DISCUSSION

HMRC works by making centroid-based routing more expressive. It does not change the within-segment search procedure, the segment structure, or the data layout. Any system that uses centroid-based routing with non-clustered segments can retrofit HMRC by adding per-segment k-means during index construction.

There are cases where HMRC provides little benefit. When segments are semantically coherent (as in a well-tuned IVF index), the method adds overhead without improving recall. When segments are purely random with no internal structure, k-means within segments discovers noise rather than topics, and the improvement is marginal. The method is most valuable in the middle ground, heterogeneous segments with discoverable internal structure, which is where production systems tend to operate.

HMRC and multi-probe search are complementary rather than competing. HMRC improves routing quality at the centroid level, reducing the number of probes needed for a given recall target. Our extended-k results show that combining HMRC with moderate multi-probe (e.g., HMRC-3 at k=15) achieves recall levels that neither approach reaches alone. The FAISS validation in Section V-D sharpens this point: because HMRC reaches the baseline's retrieval recall at a lower *nprobe* setting, the resulting search is also faster in real wall-clock terms, not just in centroid comparisons.

One limitation of this work is dataset scale. We evaluate on 100K passages; production systems serve billions of vectors. Our overhead analysis (Section IV-D) shows that HMRC's cost is proportional to the number of segments, not the corpus size, so the storage overhead argument transfers. We have not, however, measured routing quality at billion scale, where segment heterogeneity patterns may differ.

TABLE 9. Time-batched segment results (Recall@10).

Method	MS MARCO	NQ
Baseline	35.1%	21.8%
HMRC-3	53.4%	27.0%
HMRC-5	65.0%	31.9%

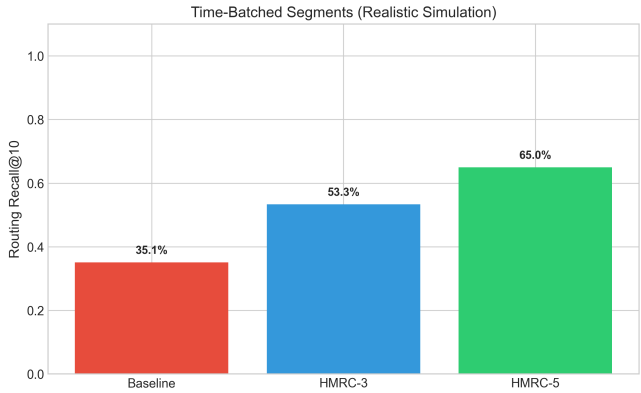

FIGURE 7. HMRC on realistic time-batched segments (MS MARCO). Sequential ingestion-order segments are highly heterogeneous. HMRC-3 improves Recall@10 from 35.1% to 53.4%; HMRC-5 reaches 65.0%.

A second limitation is the use of k-means for intra-segment clustering. K-means assumes roughly spherical clusters and is sensitive to outliers [22]. In embedding spaces produced by modern sentence transformers, topic clusters tend to be roughly isotropic after L2 normalization, making k-means a reasonable fit. Alternative clustering methods such as Gaussian mixture models or density-based approaches (DBSCAN [23]) could be explored, though they would increase per-segment build time.

Another direction worth exploring is adaptive selection of r per segment. Rather than using a fixed r for all segments, one could measure each segment's internal coherence (e.g., via silhouette score) and use more representatives only where needed. This would reduce overhead on coherent segments while focusing capacity where it matters most.

VII. CONCLUSION

We identified and quantified a gap in how vector databases handle heterogeneous segments. When segments mix three topics, routing recall drops by 36–40 percentage points across two datasets. HMRC addresses this by generating multiple representative centroids per segment using intra-segment k-means clustering. With $r = 3$ representatives, HMRC recovers most of the lost recall, improving by 26–31%, at 0.6% additional storage overhead. The method maintains 100% recall on coherent segments, confirming it does no harm when the problem does not exist. On realistic time-batched segments, the improvement is even larger.

HMRC is a drop-in enhancement. It requires no changes to segment structure, within-segment search, or data management. Validation inside FAISS IndexIVFFlat confirms that the routing improvement transfers to a production-grade index, with HMRC-3 matching the baseline's retrieval Recall@10 at roughly $4\times$ lower search latency. We recommend $r = 3$ as a robust default for most workloads. Systems operating with heterogeneous segments, whether from time-batched ingestion, multi-tenant architectures, or append-only growth, stand to gain meaningfully from this simple enhancement.

ACKNOWLEDGMENT

The author used Claude (Anthropic) for manuscript editing, grammar refinement, and assistance with experimental code implementation. All experimental design, methodology, analysis, interpretation of results, and intellectual contributions are solely the author's. AI-generated code was reviewed, tested, and verified by the author.

REFERENCES

[1] S. Guha, R. Rastogi, and K. Shim, "CURE: An efficient clustering algorithm for large databases," *ACM SIGMOD Record*, vol. 27, no. 2, pp. 73–84, 1998.

[2] L. Gottesbüren, L. Dhulipala, R. Jayaram, and J. Łącki, "Unleashing graph partitioning for large-scale nearest neighbor search," *Proc. VLDB Endow.*, vol. 18, no. 6, pp. 1649–1662, 2025.

[3] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[4] J. Wang *et al.*, "Milvus: A purpose-built vector data management system," in *Proc. ACM SIGMOD*, pp. 2614–2627, 2021.

[5] R. Guo, P. Sun, E. Lindgren, Q. Geng, D. Simcha, F. Chern, and S. Kumar, "Accelerating large-scale inference with anisotropic vector quantization," in *Proc. ICML*, pp. 3887–3896, 2020.

[6] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.

[7] Q. Lv, W. Josephson, Z. Wang, M. Charikar, and K. Li, "Multi-probe LSH: Efficient indexing for high-dimensional similarity search," in *Proc. VLDB*, pp. 950–961, 2007.

[8] R. Muja and D. G. Lowe, "Fast approximate nearest neighbors with automatic algorithm configuration," in *Proc. VISAPP*, pp. 331–340, 2009.

[9] Oracle, "Understand local inverted file flat vector indexes," *Oracle Database 26c AI Vector Search User's Guide*, 2025.

[10] J. Pan, J. Wang, and G. Li, "Survey of vector database management systems," *VLDB J.*, vol. 33, pp. 1591–1615, 2024.

[11] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, pp. 9459–9474, 2020.

[12] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnaswamy, and R. Kadekodi, "DiskANN: Fast accurate billion-point nearest neighbor search on a single node," in *Proc. NeurIPS*, pp. 13766–13776, 2019.

[13] D. Arthur and S. Vassilvitskii, "k-means++: The advantages of careful seeding," in *Proc. ACM-SIAM SODA*, pp. 1027–1035, 2007.

[14] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, 2011.

[15] C. Fu, C. Xiang, C. Wang, and D. Cai, "Fast approximate nearest neighbor search with the navigating spreading-out graph," *Proc. VLDB Endow.*, vol. 12, no. 5, pp. 461–474, 2019.

[16] Q. Chen, B. Zhao, H. Wang, M. Li, C. Liu, Z. Li, M. Yang, and J. Wang, "SPANN: Highly-efficient billion-scale approximate nearest neighbor search," in *Proc. NeurIPS*, pp. 5199–5212, 2021.

[17] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, and L. Deng, "MS MARCO: A human generated machine reading comprehension dataset," in *Proc. CoCo@NIPS*, 2016.

[18] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP*, pp. 3982–3992, 2019.

[19] T. Kwiatkowski *et al.*, "Natural Questions: A benchmark for question answering research," *Trans. Assoc. Comput. Linguist.*, vol. 7, pp. 453–466, 2019.

[20] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *Proc. NeurIPS Datasets and Benchmarks*, 2021.

[21] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[22] A. K. Jain, "Data clustering: 50 years beyond K-means," *Pattern Recognit. Lett.*, vol. 31, no. 8, pp. 651–666, 2010.

[23] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. KDD*, pp. 226–231, 1996.

[24] W. Li, Y. Zhang, Y. Sun, W. Wang, M. Li, W. Zhang, and X. Lin, "Approximate nearest neighbor search on high dimensional data: Experiments, analyses, and improvement," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 8, pp. 1475–1488, 2020.